

Projeto 1



1.Introdução e Objetivo

O e-Commerce é uma das formas mais bem-sucedidas de realizar transações comerciais a nível mundial, exigindo plataformas capazes de suportar um número elevado de utilizadores, compras simultâneas e operações críticas, como pagamentos, encomendas e gestão de stock. Sistemas de e-Commerce de referência, como a Amazon, usam uma arquitetura distribuída, na qual diferentes perfis de utilizador podem comunicar com um (ou mais) servidor(es).

Neste projeto pretende-se desenvolver uma aplicação distribuída de e-commerce, chamada *MarketCenter*, composta por um servidor central e vários componentes distribuídos que simulam os principais blocos de uma plataforma de comércio online: o funcionário (administrador ou funcionário comum) e o cliente final (registado ou anónimo). A aplicação deverá permitir que o sistema faça a gestão de produtos, de stock, carrinho e encomendas.

2.Avaliação

A avaliação prática da disciplina de Aplicações Distribuídas (AD) será dividida em **três fases**. As três fases terão continuidade entre si. Desta forma, cada nova entrega irá expandir o trabalho anterior, resultando num trabalho final mais integrado. Ao longo do semestre, os estudantes irão desenvolver uma aplicação distribuída, aplicando conceitos como comunicação, sincronização, tolerância a falhas, escalabilidade e segurança.

Na Tabela 1, apresentam-se os objetivos do projeto a serem alcançados em cada uma das fases.

Objectivo	Fase 1	Fase 2	Fase 3
Lógica de Negócio	X	X	X
Sockets TCP	X	X	X
Mensagens Texto Livre	X		
Suporte Mensagens Fragmentadas		X	X
Estruturação RPC de Comunicações		X	X
Multiplexação de I/O		X	X
Tratamento de Erros	Básico	Intermédio	Avançado
Múltiplos Clientes em Simultâneo	Não	Sim	Sim

Persistência de Dados com pickle		X	
Gestão de Base de Dados			X
Protocolo HTTP			X
Servidor <i>Flask</i>			X
Comunicação segura			X
Uso de API REST externa (Loja da Amazon)			X

Tabela 1: Requisitos e objetivos a cumprir em cada fase do projeto.

A presente tabela poderá sofrer alterações ao longo do semestre, caso se verifique necessidade de ajustes pedagógicos, técnicos ou de calendário. Quaisquer alterações serão devidamente comunicadas aos estudantes com a antecedência possível.

A implementação de funcionalidades adicionais numa fase em que estas não sejam solicitadas poderá influenciar negativamente a classificação final do projeto. A inclusão de requisitos fora do âmbito definido não é valorizada e pode até mesmo originar uma penalização, a ser determinada pelos docentes. Assim, é fundamental que os requisitos sejam cumpridos de forma rigorosa e estritamente de acordo com o enunciado.

O calendário a seguir apresenta a organização das três fases do projeto da unidade curricular, incluindo as datas de disponibilização dos enunciados e as respetivas datas de entrega.

Projeto	Enunciado	Data de Entrega
Fase 1	23 de fevereiro	6 de março
Fase 2	9 de março	27 de março
Fase 3	13 de abril	15 de maio

Tabela 2: Calendário de entregas de projetos e datas de enunciados.

As fases do projeto 1 e 2 não contemplam momento de discussão ou defesa oral. A avaliação é realizada exclusivamente pelo corpo docente, com base na aplicação de uma bateria de testes técnicos ao trabalho submetido e na sua própria análise crítica, considerando critérios de rigor, qualidade metodológica e adequação das soluções apresentadas. No final da Fase 2, será ainda realizado um teste individual no Moodle, com o objetivo de aferir os conhecimentos adquiridos até essa etapa. A avaliação com discussão do Projeto 3 decorrerá nas aulas TP e PL das duas semanas seguintes à respetiva entrega, com início na semana de 18 de maio (segunda-feira).

A classificação mínima da componente de avaliação dos projetos é de 9.5 valores, aplicada à média ponderada das notas dos projetos (**Fase 1: 25%; Fase 2: 25%, Fase 3: 50%**). Não há nota mínima para cada projeto. O desenvolvimento do projeto é feito em grupos de **2 alunos**. Em caso excecional de realização do trabalho por apenas um estudante, este será avaliado pelos mesmos critérios aplicáveis a um grupo de dois elementos.

Na avaliação das fases 1 e 2 e também na discussão final (Avaliação do Projeto), serão consideradas as impressões dos professores nas aulas, bem como a apreciação e a discussão dos resultados dos trabalhos (se aplicável).

3.Fase 1

3.1. Objetivo

Nesta primeira fase do projeto, vamos focar-nos no estabelecimento do modelo cliente-servidor **sem persistência**, implementando a comunicação básica entre clientes e um único servidor. O sistema deverá garantir que o servidor consegue aceitar ligações (**uma de cada vez**), receber e responder a pedidos de cada cliente conectado, mantendo o estado apenas em memória durante a execução. Um objetivo principal é implementar a lógica de negócio da aplicação, disponibilizando operações essenciais de e-commerce (e.g., consulta de catálogo de produtos, criação e gestão do carrinho de compras).

Na tabela a seguir, definem-se os requisitos de implementação da **primeira fase do projeto**.

Aspeto	Requisito para a 1.º Fase do Projeto
Comunicação	A comunicação entre cliente e servidor será através de mensagens de texto livre , pela rede.
Sincronização	Assume-se um modelo sequencial de execução , processando apenas um pedido de cada vez. Nesta fase não existem acessos concorrentes ao mesmo recurso.
Tolerância a falhas	Assume-se que o servidor central está sempre disponível e que a rede é fiável.
Escalabilidade	O servidor trata um cliente de cada vez.
Segurança	Assume-se um ambiente de execução controlado e confiável.

Tabela 3: Áreas de interesse e objetivos a cumprir na primeira fase do projeto.

3.2. Modelo Cliente-Servidor

No modelo cliente-servidor, a aplicação é organizada em duas componentes principais: o programa **cliente**, que interage com os diferentes perfis de utilizador (funcionário administrador, funcionário comum, e cliente final anónimo assim como cliente registado) e envia pedidos ao servidor. O programa **servidor** é responsável pelo processamento desses pedidos e pela gestão dos dados (recursos).

Nesta fase do desenvolvimento, não é necessário que a aplicação distribuída implemente a lógica associada aos diferentes perfis de utilizador. A gestão de permissões e controlo de acesso será considerada numa fase posterior do projeto.

Nesta primeira fase do projeto, o acesso aos recursos é gerido exclusivamente pelo servidor. Isto significa que todas as operações de gestão de recursos (ex.: atualização de stock, gestão de produtos e encomendas, consulta de dados, registo de clientes finais) passam obrigatoriamente pelo servidor central. Nenhum cliente do sistema mantém localmente dados como stock, listas de produtos ou informações de outras entidades.

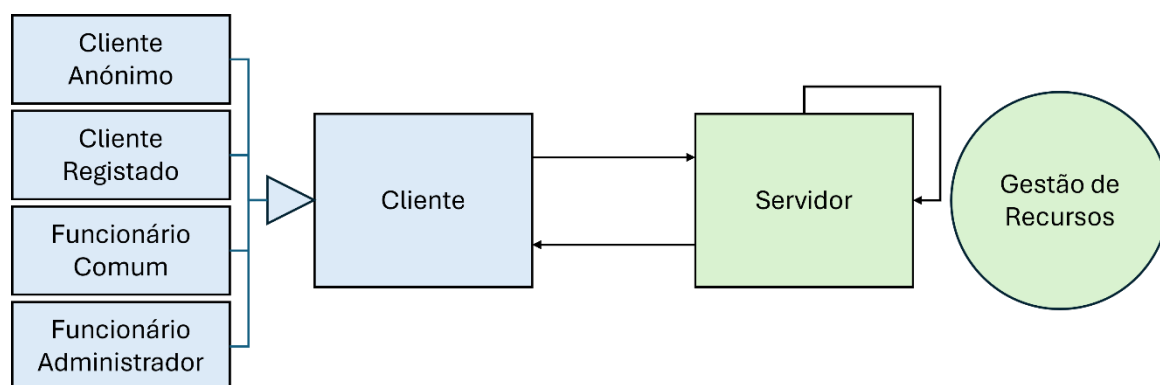


Figura 1: Modelo cliente-servidor : os perfis interagem com o sistema através de um programa Cliente, que comunica com o Servidor, que é responsável pela lógica de negócio.

3.3. Comandos e Protocolo

A comunicação entre cliente e servidor é realizada através de sockets TCP. Cada mensagem corresponde a uma linha de texto codificada em UTF-8. O primeiro elemento da mensagem identifica o nome do comando (e.g. CRIA_CATEGORIA), seguido dos respetivos argumentos separados por espaços. As respostas do servidor seguem o formato:

- OK; <resposta> — execução bem-sucedida
- NOK; <descrição do erro> — erro na execução

Os argumentos podem conter espaços (e.g., Máquinas de Lavar), devendo nesse caso ser tratados como uma única unidade lógica. **Importante: Argumentos que contenham espaços devem obrigatoriamente ser fornecidos entre aspas duplas.**

Para efeitos de comparação e verificação de unicidade, os nomes de categorias, produtos e clientes devem ser tratados de forma normalizada. A normalização consiste em:

- Remover espaços no início e no fim do texto.
- Substituir múltiplos espaços consecutivos por um único espaço.
- Ignorar diferenças entre letras maiúsculas e minúsculas.

A comparação entre nomes deve ser feita com base na versão normalizada. Para efeitos de apresentação ao utilizador, o nome deve ser armazenado com capitalização adequada das palavras.

De seguida descrevem-se os principais comandos e a lista dos perfis com permissão para executar cada comando. Quando não for referido o formato de resposta, a mensagem fica à descrição do aluno desde que cumpra o formato OK; <resposta> ou NOK; <descrição do erro>.

Na Fase 1, o sistema deve manter todos os dados exclusivamente em memória. Não é permitida qualquer forma de persistência externa. Ao reiniciar o servidor, todos os dados devem ser perdidos.

Os identificadores únicos (IDs) de categorias, produtos, clientes e encomendas devem:

- Ser numéricos.
- Ser atribuídos automaticamente pelo sistema.
- Começar em 1.

- Ser incrementais e sequenciais (1, 2, 3, 4, ...).
- Não devem ser reutilizados, mesmo que a entidade correspondente seja removida.

Todas as respostas de sucesso devem ser precedidas por OK; incluindo mensagens como “Sem Categorias.”, “Sem Produtos.”, etc. As mensagens de insucesso devem ser precedidas por NOK; e devem ser estruturadas de forma coerente.

Para efeitos deste projeto, considera-se que o arredondamento deve ser realizado utilizando a função *round(valor, 2)* da linguagem Python, garantindo que todos os preços são apresentados com duas casas decimais. Embora esta função utilize o método *round half to even* (arredondamento para o par mais próximo), para simplificação e uniformização da avaliação é obrigatório que os estudantes utilizem explicitamente *round(..., 2)* sempre que seja necessário arredondar valores monetários. O valor deve ser apresentado formatado com exatamente duas casas decimais (ex.: 2.50 e não 2.5).

Propriedade	Valor
CATEGORIAS	
Nome Comando	CRIA_CATEGORIA
Argumentos	CRIA_CATEGORIA <nome_categoria>
Exemplo Comando	CRIA_CATEGORIA Fruta
Descrição	Regista uma nova categoria no sistema, garantindo que o nome é único, e atribui um identificador único (ID). O sistema deve ainda guardar uma versão normalizada do nome, aplicando: • remoção de espaços redundantes (trim + colapsar múltiplos espaços internos), • comparação ignorando maiúsculas/minúsculas (para garantir unicidade), • capitalização em formato Title Case (primeira letra de cada palavra em maiúscula e restantes em minúscula). Ex.: " oCeAn pOlluTion " normalizado: "Ocean Pollution" e não pode existir outra categoria com nome equivalente (ex.: "ocean pollution", "Ocean Pollution").
Formato Resposta	Categoria <nome_categoria_normalizado> criada com sucesso.
Regras Negócio	Se já existir uma categoria com nome equivalente (após normalização), o servidor deve devolver erro a indicar que a categoria já existe.
Perfis	Administrador
Nome Comando	LISTA_CATEGORIAS
Argumentos	Sem argumentos
Descrição	Lista todas as categorias de produtos registadas no sistema. Para cada categoria, apresenta: • identificador único (ID); • nome da categoria; • número total de produtos associados (<nr_produtos_categoria>). As categorias devem ser apresentadas por ordem crescente do ID. Regras de formatação: • Cada categoria deve ser apresentada numa única linha; • Cada linha deve terminar com ponto e vírgula (;);
Formato Resposta Padrão	Total Categorias: <nr_total_categorias> Total Produtos: <nr_total_produtos> <id_categoria> - <nome_categoria> (<nr_produtos_categoria> produtos); <id_categoria> - <nome_categoria> (<nr_produtos_categoria> produtos);
Exemplo Resposta	Sem Categorias

Propriedade	Valor
CATEGORIAS	
Exemplo Resposta	Total Categorias: 4 Total Produtos: 69 1 - Fruta (10 produtos); 2 - Eletrodomésticos (0 produtos); 3 - Livros (58 produtos); 4 - Talho (1 produtos);
Regras Negócio	Caso não existam categorias, o servidor deve retornar a mensagem de OK com o texto “Sem Categorias”.
Perfis	Todos
Nome Comando	REMOVE_CATEGORIA
Argumentos	REMOVE_CATEGORIA <nome_categoria>
Exemplo Comando	REMOVE_CATEGORIA Fruta
Descrição	Remove uma categoria do sistema, desde que não existam produtos associados com quantidade disponível superior a zero. A verificação da existência da categoria deve ser feita usando a versão normalizada do nome.
Formato Resposta	Categoria <nome_categoria_normalizado> removida com sucesso.
Regras Negócio	Caso exista pelo menos um produto associado com quantidade superior a zero, o servidor deve retornar uma mensagem de erro indicando que a categoria contém produtos com quantidade disponível superior a zero. Caso a categoria não exista no sistema, deve ser gerada uma mensagem de erro indicando que a categoria não existe.
Perfis	Administrador

Tabela 4 - Comandos de gestão da categoria de produto.

Propriedade	Valor
PRODUTOS	
Nome Comando	CRIA_PRODUTO
Argumentos	CRIA_PRODUTO <nome_produto> <nome_categoria> <preco> <quantidade>
Descrição	Regista um novo produto no sistema, associando-o a uma categoria existente, com um determinado nome, preço e quantidade inicial. O sistema deve ainda guardar uma versão normalizada do nome, semelhante ao caso CRIA_CATEGORIA.
Formato Resposta	Produto <nome_produto_normalizado> criado com sucesso.
Regras Negócio	O preço deve ser um valor numérico positivo. A quantidade deve ser um número inteiro maior ou igual a zero. Caso o nome do produto já exista, a categoria não exista, ou os valores sejam inválidos, deve ser gerada uma mensagem de erro indicando a razão da falha. O preço deve ser armazenado e apresentado com duas casas decimais. O nome do produto deve ser único no sistema, sendo a comparação feita usando a versão normalizada do nome.
Perfis	Administrador, Funcionário Comum
Nome Comando	LISTA_PRODUTOS
Argumentos	Sem argumentos.
Descrição	Lista todos os produtos registados no sistema, apresentando o identificador único (ID), o nome do produto, sua categoria, preço e quantidade disponível. Os produtos devem ser apresentados na ordem crescente do seu ID. Cada produto apresentado termina com ponto e vírgula (;). A variável <categoria> corresponde ao nome da categoria associada ao produto.
Formato Resposta Padrão	Total Produtos: <nr_total_produtos> Total Quantidade: <quantidade_total_produtos>

Propriedade	Valor
PRODUTOS	
	<id_produto> - <nome_produto> (<nome_categoria>, <preco> euros, <quantidade> unidades);
Exemplo	Total Produtos: 3 Total Quantidade: 42 101 - Arroz Integral (Alimentação, 2.49 euros, 15 unidades); 102 - Leite Meio-Gordo (Laticínios, 0.89 euros, 20 unidades); 103 - Detergente Líquido (Limpeza, 4.75 euros, 7 unidades);
Exemplo	Sem Produtos.
Regras Negócio	Caso não existam produtos registados no sistema, deve ser apresentada a mensagem de OK indicando o texto “Sem Produtos.”. O preço deve ser apresentado com duas casas decimais.
Perfis	Todos
Nome Comando	AUMENTA_STOCK_PRODUTO
Argumentos	AUMENTA_STOCK_PRODUTO <nome_produto> <delta_quantidade>
Descrição	Aumenta o stock do produto (pelo valor em delta_quantidade). Especificamente, incrementa a quantidade disponível de um produto pelo valor especificado em <delta_quantidade>.
Formato Resposta	Stock do produto <nome_produto> aumentado em <delta_quantidade> unidades com sucesso.
Regras Negócio	Caso o produto não exista, deve ser gerada uma mensagem de erro. <delta_quantidade> deve ser um número inteiro positivo (> 0). Caso contrário, deve ser gerada mensagem de erro. A verificação da existência do produto deve ser feita usando a versão normalizada do nome.
Perfis	Administrador, Funcionário Comum
Nome Comando	ATUALIZA_PRECO_PRODUTO
Argumentos	ATUALIZA_PRECO_PRODUTO <nome_produto> <novo_preco>
Descrição	Atualiza o preço de um produto existente no sistema.
Formato Resposta	O preço do produto <nome_produto> foi atualizado para <novo_preco> com sucesso.
Regras Negócio	O produto indicado deve existir no sistema. <novo_preco> deve ser um valor real superior a zero. O preço deve ser armazenado e apresentado com duas casas decimais. Caso alguma das condições anteriores não seja satisfeita, deve ser gerada uma mensagem de erro indicando a causa. A verificação da existência do produto deve ser feita usando a versão normalizada do nome.
Perfis	Administrador, Funcionário Comum

Tabela 5 - Comandos de gestão de produto.

Propriedade	Valor
CLIENTES	
Nome Comando	CRIA_CLIENTE
Argumentos	CRIA_CLIENTE <nome_cliente> <email> <password>
Descrição	Regista um novo cliente no sistema, criando uma conta associada a um nome, email e palavra-passe.
Formato Resposta	Cliente criado com sucesso com identificador único <id_cliente>.
Regras Negócio	O email deve ser único no sistema (não pode existir outro cliente com o mesmo email). O nome do cliente não necessita de ser único.

Propriedade	Valor
CLIENTES	
	Não existem validações adicionais nesta fase relativamente ao formato do email ou robustez da password. O email deve ser comparado ignorando maiúsculas/minúsculas. O nome do cliente deve ser armazenado numa versão normalizada (remoção de espaços redundantes e capitalização adequada). A password é armazenada tal como fornecida (não é aplicada encriptação nesta fase).
Perfis	Cliente Anónimo
Comando	LISTA_CLIENTES
Argumentos	Sem argumentos.
Descrição	Lista todos os clientes registados no sistema, apresentando: • Identificador único do cliente (ID) • Nome do cliente • Email do cliente
Formato Resposta Padrão	Total Clientes: <nr_total_clientes> <id_cliente> - <nome_cliente> (<email>);
Exemplo	Total Clientes: 2 1 - Maria Silva (maria.silva@email.com); 2 - Ana Silva (ana.silva@email.com);
Exemplo	Sem Clientes
Regras Negócio	Os clientes devem ser apresentados por ordem crescente do seu ID. Caso não existam clientes registados no sistema, deve ser apresentada a mensagem “Sem Clientes”. Cada cliente apresentado termina com ponto e vírgula (;).
Perfis	Administrador, Funcionário Comum

Tabela 6 - Comandos de gestão de cliente.

Propriedade	Valor
CARRINHO DE COMPRAS	
Nome Comando	ADICIONA_PRODUTO_CARRINHO
Argumentos	ADICIONA_PRODUTO_CARRINHO <id_cliente> <nome_produto> <quantidade>
Descrição	Adiciona um determinado produto ao carrinho de compras do cliente.
Formato Resposta	Produto <nome_produto_normalizado> adicionado com sucesso ao carrinho.
Regras Negócio	O cliente deve estar registado no sistema. O produto deve existir. A verificação da existência do produto deve ser feita usando a versão normalizada do nome. Se o produto já existir dentro do carrinho: • Incrementar a quantidade existente por <quantidade>. • Aplicar novamente validação de stock. A quantidade deve ser um número inteiro maior que zero. A quantidade solicitada não pode ser superior à quantidade disponível em stock. O stock do produto deve ser reduzido pela quantidade adicionada ao carrinho. Caso alguma condição das prévias não seja satisfeita, deve ser gerada mensagem de erro indicando a causa. O aumento de stock deve ocorrer apenas após todas as validações serem satisfeitas.
Perfis	Cliente Registado

Propriedade	Valor
CARRINHO DE COMPRAS	
Nome Comando	REMOVE_PRODUTO_CARRINHO
Argumentos	REMOVE_PRODUTO_CARRINHO <id_cliente> <nome produto>
Descrição	Remove um produto do carrinho do cliente.
Formato Resposta	Produto <nome_produto_normalizado> removido com sucesso do carrinho de compras.
Regras Negócio	O cliente deve estar registado. O produto deve existir. A verificação da existência do produto deve ser feita usando a versão normalizada do nome. O produto deve estar presente no carrinho do cliente. O produto deve ser removido totalmente do carrinho, sendo toda a quantidade previamente existente reposta ao stock. Caso alguma condição não seja satisfeita, deve ser gerada mensagem de erro indicando a causa. A reposição de stock deve ocorrer apenas após todas as validações serem satisfeitas.
Perfis	Cliente Registado
Nome Comando	LISTA_CARRINHO
Argumentos	LISTA_CARRINHO <id_cliente>
Descrição	Lista os produtos atualmente presentes no carrinho do cliente.
Formato Resposta	Total Produtos: <nr_total_produtos> Total Quantidade: <quantidade_total> Total Preço: <preco_total> euros <id_produto> - <nome_produto> (<id_categoria>-<nome_categoria>, <preco_unitario> euros, <quantidade> unidades);
Exemplo	Total Produtos: 3 Total Quantidade: 8 Total Preço: 154.92 euros 101 - Arroz Integral (1-Alimentação, 2.49 euros, 4 unidades); 205 - Detergente Líquido (3-Limpeza, 4.75 euros, 2 unidades); 310 - Café Torrado (2-Bebidas, 69.99 euros, 2 unidades);
Exemplo	Carrinho Vazio
Regras Negócio	O cliente deve estar registado. Caso o carrinho esteja vazio, deve ser apresentada a mensagem “Carrinho Vazio”. O preço deve ser apresentado com duas casas decimais. Os produtos devem ser apresentados por ordem crescente de ID. Caso o cliente não exista, deve ser gerada mensagem de erro apropriada. Quando aparece “Carrinho Vazio”, não aparecem os totais. Cálculo dos totais: - Total Produtos: <nr_total_produtos> corresponde ao número de produtos distintos no carrinho (número de linhas listadas). - Total Quantidade: <quantidade_total> corresponde à soma das quantidades de todos os produtos no carrinho. - Total Preço: <preco_total> euros corresponde à soma de (preço unitário × quantidade) para todos os produtos do carrinho, ou seja: $\text{preco_total} = \sum_i (\text{preco_unitario}_i \times \text{quantidade}_i)$
Perfis	Cliente Registado
Nome Comando	CHECKOUT_CARRINHO

Propriedade	Valor
CARRINHO DE COMPRAS	
Argumentos	CHECKOUT_CARRINHO <id_cliente>
Descrição	Cria uma nova encomenda a partir dos produtos presentes no carrinho do cliente. Deve registar a data da nova encomenda.
Formato Resposta	Checkout de carrinho de compras efetuado com sucesso. Encomenda criada com sucesso a partir do carrinho.
Regras Negócio	O cliente deve estar registado. O carrinho deve conter pelo menos um produto. Deve ser criada uma nova encomenda. O total da encomenda deve corresponder à soma (preço × quantidade) dos produtos do carrinho. Após o checkout, o carrinho deve ser esvaziado. O checkout não deve alterar o stock, uma vez que este já foi ajustado no momento da adição ao carrinho. Caso o carrinho esteja vazio, deve ser gerada mensagem de erro. O preço considerado para cada produto numa encomenda deve corresponder ao preço do produto no momento do checkout do carrinho. Deve ser atribuído um identificador único à nova encomenda. Em resumo: 1. Validar cliente 2. Validar carrinho não vazio 3. Criar encomenda 4. Atribuir ID 5. Calcular total 6. Esvaziar carrinho 7. (Não alterar stock)
Perfis	Cliente Registado

Tabela 7 - Comandos de gestão de carrinho de compras.

Propriedade	Valor
ENCOMENDAS	
Nome Comando	LISTA_ENCOMENDAS
Argumentos	LISTA_ENCOMENDAS <id_cliente>
Descrição	Lista todas as encomendas associadas a um cliente específico. As encomendas devem ser apresentadas na ordem crescente do seu ID. O <nr_total_produtos_encomendas> é o número de produtos diferentes na encomenda. A data da encomenda deve ser formatada de acordo com YYYY-MM-DD HH:MM:SS.
Formato Resposta	Cliente: <nome_cliente> <email_cliente> Total Encomendas: <nr_encomendas_cliente> Total Produtos: <nr_total_produtos_encomendas> Total Preço: <preco_total_produtos_encomendas> Categoria Top: Alimentação, Bebidas ID Encomenda: <id_encomenda> Data Encomenda: <data_encomenda> Total Produtos: <nr_total_produtos> Total Quantidade: <quantidade_total> Total Preço: <preco_total> euros <id_produto> - <nome_produto> (<nome_categoria>, <preco_unitario> euros, <quantidade> unidades);

Propriedade	Valor
ENCOMENDAS	
Exemplo Resposta	Cliente: Marilia Silva marilia.silva@mail.com Total Encomendas: 1 Total Produtos: 3 Total Preço: 159.44 euros Categoria Top: Alimentação, Bebidas ----- ID Encomenda: 1 Total Produtos: 3 Total Quantidade: 8 Total Preço: 159.44 euros 101 - Arroz Integral (Alimentação, 2.49 euros, 4 unidades); 205 - Detergente Líquido (Limpeza, 4.75 euros, 2 unidades); 310 - Café Torrado (Bebidas, 69.99 euros, 2 unidades);
Exemplo Resposta	Sem Encomendas
Regras Negócio	Caso não existam encomendas registadas no sistema para o cliente, deve ser apresentada a mensagem de OK indicando o texto “Sem Encomendas”. O cliente indicado deve existir no sistema. Caso o cliente não exista, deve ser gerada uma mensagem de erro indicando que o cliente não existe. O montante total corresponde à soma do valor de todos os produtos incluídos na encomenda (preço × quantidade). O total deve ser apresentado com duas casas decimais. Categoria Top corresponde às categorias presentes na(s) encomenda(s), que tiveram mais unidades vendidas para aquele cliente. Em caso de empate, apresentar as categorias empatadas por ordem alfabética, separadas por vírgula. O total deve ser calculado com os valores já arredondados a duas casas decimais.
Perfis	Administrador, Funcionário Comum, Cliente Registrado

Tabela 8 - Comandos de gestão de encomenda.

4. Requisitos e Recomendações

4.1. Código Base

Será disponibilizado código base a partir do qual os estudantes deverão desenvolver o projeto. Recomenda-se a leitura atenta do ficheiro README.txt, onde se encontram descritas as instruções de execução, estrutura do projeto e requisitos a cumprir.

A solução deve ser construída sobre o código base fornecido, mantendo obrigatoriamente a sua estrutura original (organização de ficheiros, nomes de módulos e assinaturas das funções). Não devem ser removidas funcionalidades já existentes nem alterado o comportamento esperado das interfaces fornecidas, salvo indicação explícita no enunciado.

Qualquer submissão que altere o ficheiro testes.py poderá ser penalizada e/ou considerada inválida para efeitos de correção automática. O mesmo se aplica a alterações que comprometam a execução dos testes fornecidos.

4.2. Estruturação de Código

No âmbito desta primeira fase do projeto (e nas seguintes também), pretende-se uma implementação simples, explícita e coerente com os objetivos introdutórios da unidade curricular. Assim, não é

permitida a utilização de construções avançadas da linguagem que ocultem a lógica fundamental ou introduzam complexidade desnecessária. Especificamente, não é permitida a utilização de:

- **Anotações de tipo (*type hints*)** introduzem uma camada adicional de formalismo na definição de funções e classes.
- **Operador de união de tipos (`|`)**, como em `str | None`, é uma funcionalidade moderna que depende de versões recentes do Python e adiciona complexidade desnecessária num projeto introdutório.
- **`@dataclass`** automatiza a criação de métodos como o construtor, escondendo detalhes importantes sobre inicialização de objetos. Num projeto inicial, é preferível que o aluno escreva o `__init__` para compreender o processo de criação de instâncias.
- **Decoradores (`@algo`)**, incluindo `@property`, modificam o comportamento de funções e atributos de forma implícita. O uso de `@staticmethod` é permitido.
- **`match/case` (*pattern matching*)** é uma construção moderna que, embora útil, não é essencial para compreender estruturas de decisão básicas e pode criar dependência de versões específicas do Python.
- **Operador walrus (`:=`)** permite atribuições dentro de expressões, tornando o código mais compacto, mas também menos claro para iniciantes.
- **`eval()` e `exec()`** executam código dinamicamente, o que introduz riscos de segurança e dificulta a análise estática do programa. Além disso, afastam o foco do controlo explícito da lógica.
- **`getattr()` e `setattr()` dinâmicos** permitem metaprogramação, mas tornam o comportamento do programa menos previsível e mais difícil de seguir.
- **Funções `lambda` e compreensões complexas** podem tornar o código mais compacto, mas também menos legível quando usadas excessivamente, o que não é desejável nestes projetos. Em casos excecionais de necessidade de ordenação, podem ser utilizadas as funções `lambda`.

Em suma, todas estas construções não são “erradas”, mas introduzem níveis adicionais de abstração que podem ocultar os conceitos fundamentais que se pretendem consolidar neste primeiro projeto: comunicação por sockets, controlo explícito do fluxo de execução e estruturação clara do código.

5. Entrega

A entrega da fase 1 do projeto tem de ser feita de acordo com as seguintes regras:

- Colocar todos os ficheiros do projeto, bem como um ficheiro README, num único ficheiro ZIP. O nome do ficheiro será AD-XX-fase1.zip (XX é o número do grupo).
- Submeter o ficheiro AD-XX-fase1.zip na página da disciplina no Moodle de Ciências@ULisboa, utilizando a atividade disponibilizada para tal. Apenas um dos elementos do grupo deve submeter, considerando-se a submissão mais recente, caso existam várias.

O ficheiro ZIP deverá conter:

- o ficheiro README, onde os alunos devem incluir informações que julguem necessárias (e.g., limitações na implementação, como executar o código, testes executados);
- uma diretoria para o código do servidor designada por **servidor**, contendo todos os ficheiros necessários à execução do Servidor (incluindo o ficheiro com a main do servidor designado main.py).
- uma diretoria para o código do cliente designada por **cliente**, contendo todos os ficheiros necessários à execução do Cliente (incluindo o ficheiro com a main do cliente designado main.py).
- ficheiro de testes denominado exatamente **testes.py** na raiz do projeto
- outras diretorias de interesse
- ficheiro **processador.py** na pasta **servidor** que contenha a **classe Processador**. É obrigatória a implementação da função **processar_comando** dentro desta classe. Esta função deve:
 - Receber uma **string** correspondente ao comando enviado pelo cliente;
 - Processar o comando de acordo com as regras do sistema;
 - Devolver uma **string de resposta** ao servidor de resposta.

Esta **string de resposta** devolvida deve obrigatoriamente seguir o protocolo definido, incluindo:

- **OK**; seguido da mensagem formatada segundo o protocolo, quando o comando é executado com sucesso. Veja regras de formatação acima;
- **NOK**; seguido da mensagem de erro formatada segundo o protocolo, quando ocorre alguma situação inválida (por exemplo, cliente inexistente, comando inválido, erro de validação, etc.). Veja regras de formatação acima.

Qualquer implementação que não respeite a existência desta função nesta classe assim como formato de resposta definido no protocolo poderá ser penalizada e/ou **considerada inválida para efeitos de correção automática**.

Todos os ficheiros entregues devem começar com um cabeçalho de comentários, no qual devem constar o número do grupo, o nome e o número dos seus elementos, e a descrição para a qual o ficheiro/código serve. Os programas são testados no ambiente Linux dos laboratórios das aulas, pelo que se recomenda que os alunos testem os seus programas neste mesmo ambiente.

O prazo de entrega é dia 6/3/2026, até às 23:59 horas.

Após esta data, a submissão do trabalho através do Moodle deixará de ser permitida.

Não são aceites entregas por email.

6.Plágios

Não é permitido aos alunos partilhar códigos com soluções, ainda que parciais, de qualquer parte do projeto com outros alunos (nem através do Fórum da disciplina, nem por qualquer outro meio). Além disso, todos os códigos serão submetidos a um verificador de plágio. Caso seja encontrada alguma irregularidade, os projetos de todos os alunos envolvidos serão anulados e o caso será reportado aos órgãos responsáveis em Ciências@ULisboa.

Chamamos a atenção para o facto de as plataformas generativas baseadas em Inteligência Artificial (e.g., o ChatGPT e o GitHub Co-Pilot) (1) gerarem um número limitado de soluções diferentes para o mesmo problema, (2) não resolverem corretamente as alíneas descritas no projeto, e (3) incluírem padrões característicos deste tipo de ferramenta, os quais podem vir a ser detetáveis pelos verificadores de plágio. Desta forma, recomendamos fortemente que os alunos não submetam trechos de código gerados por este tipo de ferramenta a fim de evitar riscos desnecessários.

Por fim, é responsabilidade de cada aluno garantir que a sua *home*, as suas diretorias e os seus ficheiros de código estão protegidos contra a leitura de outras pessoas (que não o utilizador dono dos mesmos). Por exemplo, se os ficheiros estiverem gravados na sua área de aluno nos servidores de Ciências@ULisboa, então todos os itens mencionados anteriormente devem ter as permissões de acesso 700. Se os ficheiros estiverem no GitHub, garantam que o conteúdo do vosso repositório não esteja visível publicamente. Caso contrário, a sua participação num eventual plágio será considerada ativa.